

Concurrency

Module 4

Critical Section Problem

Hardware Support for Synchronization

Critical Section Problem

Hardware Support for Synchronization

Hardware-based solutions for critical sections leverage **specialized instructions provided by the processor** to ensure mutual exclusion.

These instructions are **atomic**, meaning they execute as a single, indivisible operation, preventing other processes from interfering.

Critical Section Problem

- Does this scheme provide mutual exclusion?

Process 1

```
while(1){  
    while(lock != 0);  
    lock = 1; // lock  
    critical section  
    lock = 0; // unlock  
    other code  
}
```

lock=0

Process 2

```
while(1){  
    while(lock != 0);  
    lock = 1; // lock  
    critical section  
    lock = 0; // unlock  
    other code  
}
```

Critical Section Problem

- Does this scheme provide mutual exclusion?

Process 1

```
while(1){  
    while(lock != 0);  
    lock = 1; // lock  
    critical section  
    lock = 0; // unlock  
    other code  
}
```

lock=0

Process 2

```
while(1){  
    while(lock != 0);  
    lock = 1; // lock  
    critical section  
    lock = 0; // unlock  
    other code  
}
```

No

lock = 0

P1: while(lock != 0);

P2: while(lock != 0);

P2: lock = 1;

P1: lock = 1;

.... Both processes in critical section

context switch

Critical Section Problem

Synchronization Hardware 1 – Test and Set

- It's a locking mechanism that prevents multiple processes from accessing the same resource
- A **shared Boolean variable**, commonly known as a '**lock**', is used.
- When a process wants to access a resource, it 'tests' the lock. If the lock is **false (unlocked)**, the process sets it to **true (locked)** and proceeds with its task. If the lock is true, the process waits until it's released.

An Example

```
boolean TestAndSet (boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

```
do {  
    while (TestAndSet (&lock) );  
    // do nothing  
    // critical section  
    lock = FALSE;  
    // remainder section  
} while (TRUE);
```



```
do {  
    while (TestAndSet (&lock) );  
    // do nothing  
    // critical section  
    lock = FALSE;  
    // remainder section  
} while (TRUE);
```

Critical Section Problem

Synchronization Hardware 1 - **Test and Set**

```
boolean lock = false;
boolean TestAndSet(boolean &target){
    boolean returnValue = target;
    target = true;
    return returnValue;
}
while(1){
    while(TestAndSet(lock));
    CRITICAL SECTION CODE;
    lock = false;
    REMAINDER SECTION CODE;
}
```


Critical Section Problem

Synchronization Hardware 1 – Test and Set

Thread 1:

- Enters the outer while(1) loop.
- Enters the inner while(TestAndSet(lock)) loop.
- TestAndSet sets lock to true and returns false, exiting the inner loop.
- Executes the CRITICAL SECTION CODE.
- Sets lock to false.
- Executes the REMAINDER SECTION CODE.
- Goes back to the beginning of the outer loop.

Synchronization Hardware 1 – Test and Set

Thread 2:

- Enters the outer while(1) loop.
- Enters the inner while(TestAndSet(lock)) loop.
- TestAndSet sees that lock is already true and returns true, continuing the inner loop.
- Thread 2 will keep spinning in this loop until Thread 1 releases the lock.

Critical Section Problem

Synchronization Hardware 1 – **Test and Set**

- The target parameter in the function is passed by reference, meaning that any changes made to target within the function will also affect the original variable that was passed to the function. In this case, the original variable is lock.

Critical Section Problem

Synchronization Hardware 2 – Compare and Swap

It involves two operations: 'swap' and 'unlock'.

A boolean lock variable is used here as well.

The 'swap' operation exchanges the value of the lock with a local variable.

If the lock was false, it becomes true, and the process proceeds.

The 'unlock' operation sets the lock back to false when the process is done.

Synchronization Hardware 2 – Compare and Swap

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;

    if (*value == expected)
        *value = new_value;

    return temp;
}

while (true) {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

    /* critical section */

    lock = 0;

    /* remainder section */
}
```

*// Mutual exclusion with the
compare and swap() instruction.*

Figure Mutual exclusion with the compare and swap() instruction

Critical Section Problem – Compare and Swap

Synchronization Hardware 2 – Swap

```
boolean lock = false;  
individual key = false;
```

```
void swap(boolean &a, boolean &b){  
    boolean temp = a;  
    a = b;  
    b = temp;  
}
```

```
while(1){  
    key=true;  
    while(key==true)  
    {  
        swap(lock,key);  
    }
```

CRITICAL SECTION CODE

```
lock = false;
```

REMAINDER SECTION CODE

Critical Section Problem

Synchronization Hardware – Compare and Swap

- Each process executes the following loop:
 - Sets its **key to true**, indicating its intention to enter the critical section.
 - While **key** is still **true**, repeatedly swaps the values of **lock** and **key**.
 - If the **swap operation** is **successful** (i.e., **lock** was **false** and **key** was **true**), the process has acquired the lock and can enter the critical section.
 - After executing the critical section, the process sets **lock** to **false** to release the lock for other processes.
 - The loop continues, allowing the process to try entering the critical section again if necessary.

Critical Section Problem

Synchronization Hardware – Compare and Swap

- The `compare_and_swap` instruction attempts to atomically update the lock variable.
- If the lock is currently 0, it is set to 1, indicating that the process has entered the critical section.
- If the lock is already 1, the process retries the loop until it successfully acquires the lock.

Critical Section Problem

Synchronization Hardware – Compare and Swap

- Variables:
- **key:** A boolean variable for each process, indicating whether the process is trying to enter the critical section.
- **lock:** A shared boolean variable representing the lock for the critical section.

Process Synchronization – Mutex

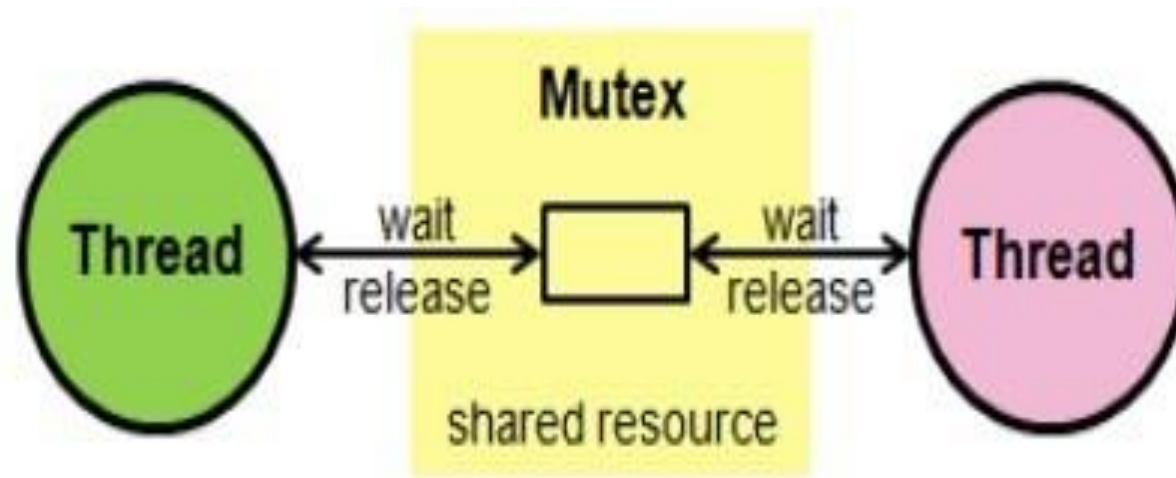
- The hardware-based solutions to the critical-section problem are complicated as well as inaccessible to application programmers.
- Operating-system designers build higher-level software tools to solve the critical-section problem, one such tool is **MUTEX**.
- The **mutex lock** is used to protect critical sections and thus prevent race conditions.
- A process must **acquire the lock** before entering a critical section; it **releases the lock** when it exits the critical section. The **acquire()** function acquires the lock, and the **release()** function releases the lock

Process Synchronization – Mutex

- A mutex lock has a **boolean variable** available whose value indicates if the lock is available or not.
- A process that attempts to acquire an unavailable lock is **blocked** until the lock is released.
- Calls to either `acquire()` or `release()` must be performed **atomically**.
- The main **disadvantage** of the implementation is that it requires **busy waiting**.
- Busy waiting wastes CPU cycles that some other process might be able to use productively

Process Synchronization – Mutex

The type of mutex lock is also called a **spinlock** because the process “spins” while waiting for the lock to become available.



Solution to the critical-section problem using mutex locks

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```

The definition of `acquire()` is as follows:

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;  
}
```

The definition of `release()` is as follows:

```
release() {  
    available = true;  
}
```

Process Synchronization – Semaphores

Process Synchronization – Semaphores

Semaphores

- A semaphore S is an integer variable that is accessed only through two standard atomic operations: `wait()` and `signal()`.
- Introduced by the Dutch scientist Edsger Dijkstra
- the `wait()` operation was originally termed P (to test) and `signal()` was originally called V (to increment)

Process Synchronization – Semaphores

Semaphores

- Used to enforce mutual exclusion, avoid race conditions, and implement synchronization between processes.
- Binary Semaphore and Counting Semaphore

Process Synchronization – Semaphores

Semaphores – Primitives - wait (P) and signal (V).

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```


Process Synchronization – Semaphores

Semaphores – Primitives - wait (P) and signal (V).

- The **wait operation** decrements the value of the semaphore, and the **signal operation** increments the value of the semaphore.
- When the value of the semaphore is zero, any process that performs a wait operation will be blocked until another process performs a signal operation.

Process Synchronization

Semaphores – Primitives - wait (P) and signal (V).

- When a process performs a wait operation on a semaphore, the operation checks whether the value of the semaphore is > 0 .
- If so, it decrements the value of the semaphore and lets the process continue its execution;
- otherwise, it blocks the process on the semaphore

Process Synchronization

Semaphores – Primitives - wait (P) and signal (V).

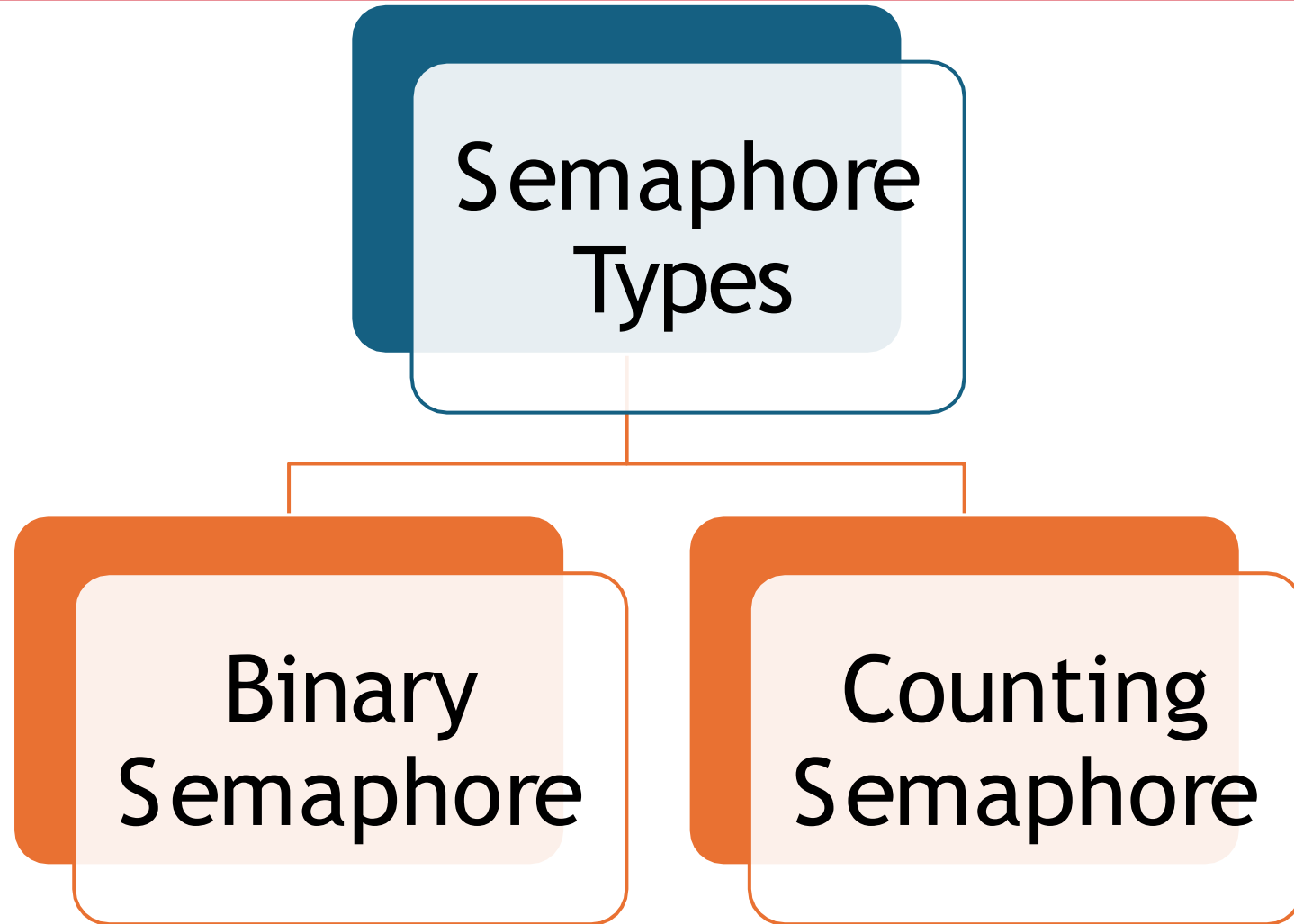
- A signal operation on a semaphore activates a process blocked on the semaphore if any or increments the value of the semaphore by 1.
- Due to these semantics, semaphores are also called counting semaphores.

Process Synchronization

Semaphores – Primitives - wait (P) and signal (V).

- **Wait()/Down()/P()**- Helps in controlling the entry of processes in CS. The operation decrements the value of its argument as soon as a process enters the critical section.
- **Signal()/UP()/V()**- Helps control the process's exit after execution from CS. Increment the value of its argument as soon as the execution is finished.

Process Synchronization



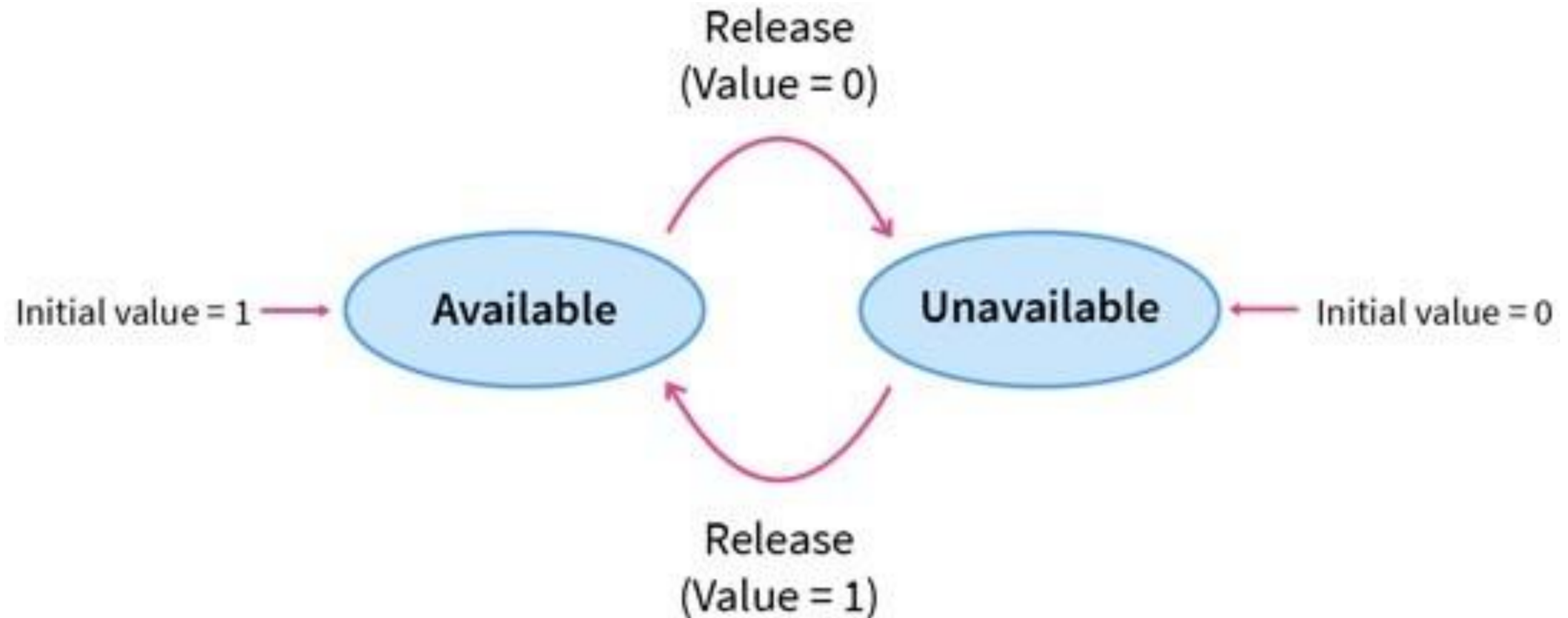
Process Synchronization

Binary Semaphores

- This is also known as a mutex lock.
- It can have only two values 0 and 1.
- Its value is initialized to 1.
- It is used to implement the solution of critical section problems with multiple processes.

Process Synchronization

Binary Semaphores



Process Synchronization

Binary Semaphores

- If the value of the semaphore was initially 1, then on the entering of the process into the critical section, the wait function would have decremented the value to 0 meaning that no more processes can access the critical section (making sure of mutual exclusion -- only in binary semaphores).

State 1 :

P1 → Executing in non critical section
S = 1
P2 → Executing in non critical section

State 2 :

P1 → Enters critical section updates
S = 0, S = 0,
P2 → Executing in non critical section

State 3 :

P1 → Executing in critical section
S = 0
P2 → Wants to enter critical section but
can't as S = 0

State 4 :

P1 → Exits critical section and updates
S = 1, S = 1
P2 → Enters critical section as S = 1 and
updates S = 0

Process Synchronization

Binary Semaphores

- Once the process exits the critical section, the signal operation is executed and the value of the semaphore is incremented by 1, meaning that the critical section can now be accessed by another process.

State 1 :

P1 → Executing in non critical section
 $S = 1$
P2 → Executing in non critical section

State 2 :

P1 → Enters critical section updates
 $S = 0$, $S = 0$,
P2 → Executing in non critical section

State 3 :

P1 → Executing in critical section
 $S = 0$
P2 → Wants to enter critical section but can't as $S = 0$

State 4 :

P1 → Exits critical section and updates
 $S = 1$ $S = 1$
P2 → Enters critical section as $S = 1$ and updates $S = 0$

Process Synchronization

Binary Semaphores

```
int semaphore = 1;
void P() {
    while (semaphore == 0) {
        // Busy waiting until the semaphore is unlocked
    }
    semaphore = 0;
}
void V() {
    semaphore = 1; // Unlock the semaphore (exit critical section)
}
```

Process Synchronization

Counting Semaphores

- Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.
- Coordinate access to resources
- Here, the Semaphore count is the number of resources available
- If the value of the Semaphore is anywhere above 0, processes can access the critical section or the shared resources.
- The number of processes that can access the resources/code is the value of the semaphore

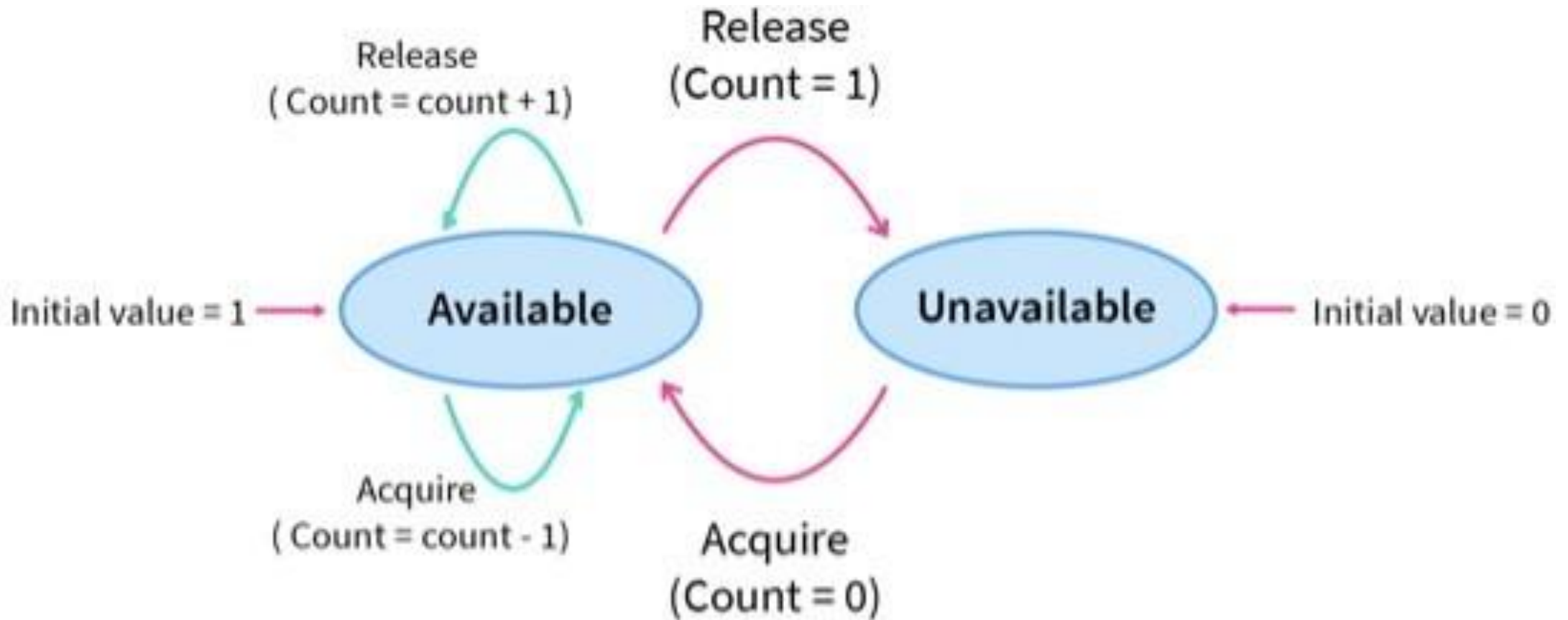
Process Synchronization

Counting Semaphores

- if the value is 0, it means that there aren't any resources that are available, or the critical section is already being accessed by several processes and cannot be accessed by more processes.
- Counting semaphores are generally used when the number of instances of a resource is more than 1, and multiple processes can access the resource.

Process Synchronization

Counting Semaphores



Process Synchronization

Counting Semaphores

- `P()` (Wait): This decreases the value of the semaphore. If the value is greater than 0, the process can proceed. If it is 0, the process must wait until the value becomes greater than 0.
- `V()` (Signal): This increases the value of the semaphore, signaling that a resource has been released and is available for other processes

Process Synchronization

Counting Semaphores

```
int semaphore = 3;
void P() {
    while (semaphore == 0) {
        // Busy waiting until a resource becomes available
    }
    semaphore--; // Decrease the semaphore value (acquire a resource)
}
void V() {
    semaphore++; // Increase the semaphore value (release a resource)
}
// Example usage with multiple processes using a shared resource
```

Process Synchronization

Semaphores Disadvantage

Busy Waiting

- it wastes CPU cycles
- It is also called as Spinlock

Process Synchronization – Monitors

Process Synchronization – Monitors

- Monitors are a programming language component that aids in the regulation of shared data access.
- The Monitor is a package that contains shared data structures, operations, and synchronization between concurrent procedure calls.

Process Synchronization – Monitors

- An **abstract data type**—or ADT—encapsulates data with a set of functions to operate on that data that are independent of any specific implementation of the ADT.
- A monitor type is an ADT that **includes a set of programmer-defined operations** that are provided with mutual exclusion within the monitor.

Process Synchronization

Why Monitor?

- Various types of errors can be generated easily when programmers use semaphores or mutex locks incorrectly
- So we need a high-level synchronization tool

Process Synchronization

Monitor

- Processes operating outside the monitor can't access the monitor's internal variables, but they can call the monitor's procedures.

```
monitor monitor_name{  
    //declaring shared variables  
  
    variables_declaration;  
    condition_variables;  
  
    procedure p1(...) {  
        ...  
    };  
  
    procedure p2(...) {  
        ...  
    };  
    ...  
    procedure pn(...) {  
        ..  
    };  
  
    {  
        Initailisation Code();  
    }  
}
```

Fig: Pseudocode syntax of a monitor

Process Synchronization

Monitor

- `monitor_name` is the name of the monitor
- `p1`, `p2`, `pn` are the procedures that provide access to shared data and variables enclosed in monitors
- It ensure that only one thread is accessed at a time for using the shared resource.

Process Synchronization

Monitor - Components

- **Initialization:** Initialization code is executed when a monitor is initialized. It is needed once when the monitors are created and initialized.
- It helps initialize any shared data structures that the monitor will use to perform specific tasks.
- Initialization code is used to initialize a variable when we want to use it.

Process Synchronization

Monitor - Components

- **Private Data:** It is an essential feature of monitors that helps make the data private.
- It is involved in holding monitors' confidential data, which includes private functions.
- Private fields and functions are not visible outside of the monitor.

Process Synchronization

Monitor - Components

- **Monitor Procedure:** Procedures operate on shared variables defined inside the monitor.
- Monitor procedures or functions can be invoked outside the monitor.
- These are functions or methods which are executed within the monitors' context.
- They are usually used for shared resource manipulation.

Process Synchronization

Monitor - Components

- **Queue:** Queue data structure maintains a list of threads waiting for a shared resource.
- Whenever we request a thread to use shared resources, but another thread is currently using the shared resource, then we cannot use the shared resource at that time.
- But whenever the resource is in its available state, the thread at the front is granted access to the resource.

Process Synchronization – About Monitor

A function defined within a monitor can access only those variables declared locally within the monitor and its formal parameters.

The monitor construct ensures that only one process at a time is active within the monitor.

In monitor, to provide synchronization additional mechanisms are provided by the condition construct.

Process Synchronization - Monitor

- **Condition Variables:** A programmer who needs to write a tailor-made synchronization scheme can define one or more variables of type condition: **condition x, y;**
- The **only operations** that can be invoked on a condition variable are wait() and signal(). The operation **x.wait();** means that the process invoking this operation is suspended until another process invokes **x.signal();**

Process Synchronization

- **Wait() Function:** This function is used for releasing a mutex associated with the monitor.
 - It waits for a particular condition to be valid.
 - A mutex is a program object that enables sharing of the same resources by multiple programs by taking turns.
 - When a thread calls the wait() function, it adds a thread to the list and puts it to sleep.
 - It releases the mutex and enters a blocked state until another thread signals the condition.
 - Therefore, variables are suspended and placed in the condition variables' block queue whenever we perform a wait operation.
- **Signal() Function:** This function is used to wake up a waiting thread.
 - When a thread calls the function, it allies the thread to proceed by waking it up.
 - This function gives a chance to one of the blocked variables.

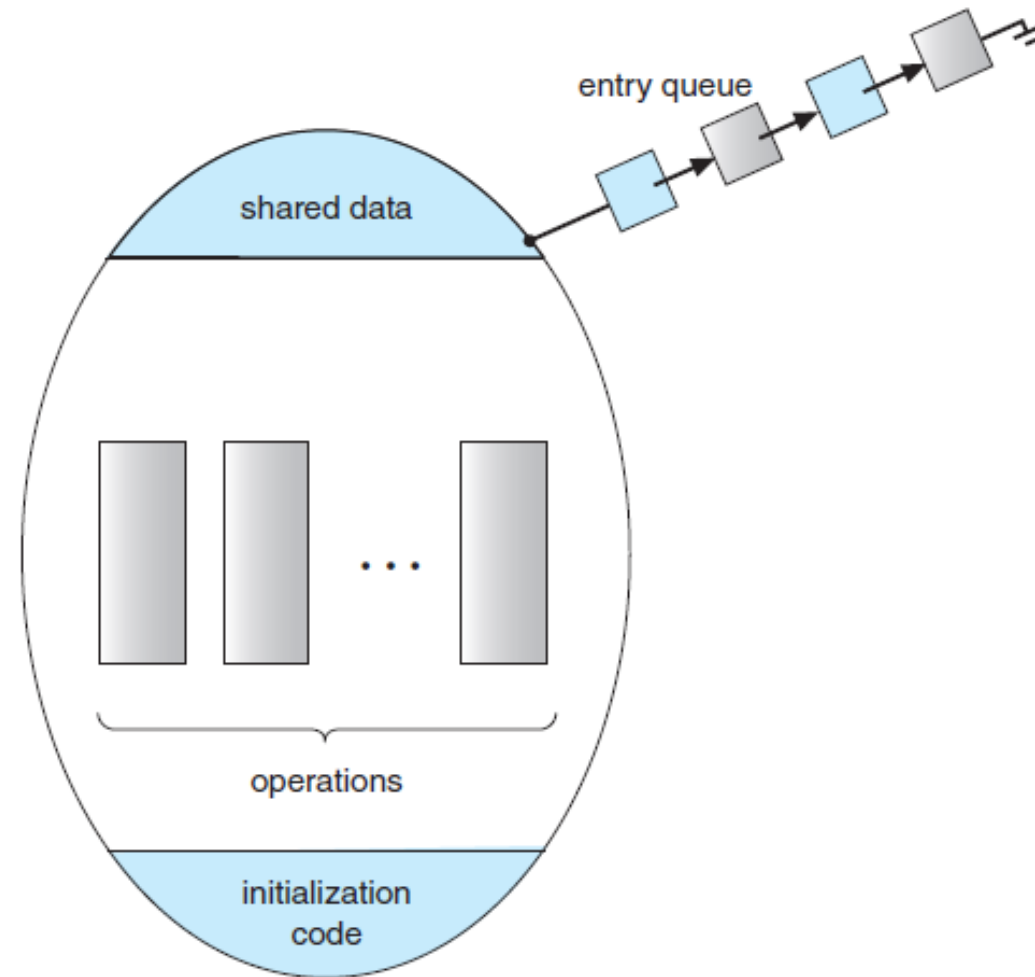
Process Synchronization

Monitor – Overall Working

- When a thread wants to access the shared data, it enters the monitor. If no other thread is using the monitor, the thread proceeds.
- If another thread is already inside the monitor, the new thread is blocked until the monitor becomes available (the current thread exits the monitor).
- Inside the monitor, condition variables can be used to let a thread wait until a specific condition is met (e.g., waiting for input or for a buffer to be filled). When this condition occurs, another thread signals it, allowing the waiting thread to proceed

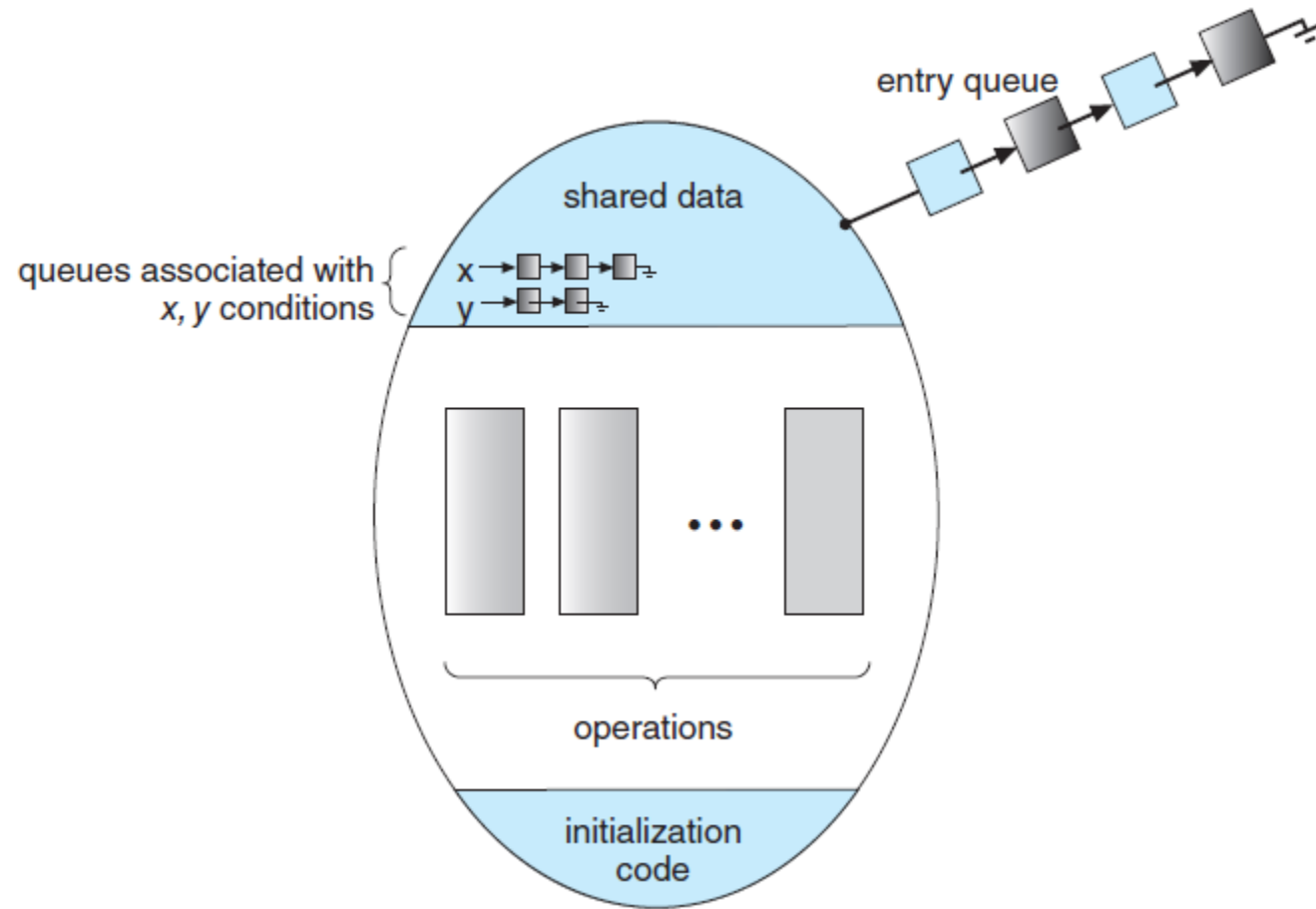
Process Synchronization

Schematic view of a monitor.



Process Synchronization

Monitor with condition variables



Implementing a Monitor Using Semaphores

Implementation of the monitor mechanism using semaphores.

- For each monitor, a binary semaphore **mutex** (initialized to 1) is provided to ensure mutual exclusion.
- A process must execute **wait(mutex)** before entering the monitor and must execute **signal(mutex)** after leaving the monitor.
- Signaling processes can use an additional binary semaphore, **next** (initialized to 0). to suspend themselves.
- An integer variable **next count** is also provided to count the number of processes suspended on next.

Resuming Processes within a Monitor

How do we determine which of the suspended processes should be resumed next? **FCFS** (or)

- the conditional wait construct can be used- **x.wait(c);**
- where c is an integer expression that is evaluated when the wait() operation is executed. The value of c, which is called a **priority number**, is then stored with the name of the process that is suspended.
- When x.signal() is executed, the process with the **smallest priority number** is resumed next.

Implementing a Monitor Using Semaphores

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;

    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = true;
    }

    void release() {
        busy = false;
        x.signal();
    }

    initialization_code() {
        busy = false;
    }
}
```

A monitor to allocate a single resource

Resuming Processes within a Monitor

How do we determine which of the suspended processes should be resumed next?

Resource allocate monitor

- Each process, when requesting an allocation of this resource, specifies the maximum time it plans to use the resource.
- The monitor allocates the resource to the process that has the shortest time-allocation request.

Resuming Processes within a Monitor

How do we determine which of the suspended processes should be resumed next?

A process that needs to access the resource in question must observe the following sequence:

R.acquire(t);

.... access the resource;

R.release();

where R is an instance of type ResourceAllocator.

Resuming Processes within a Monitor

Problems that are faced by the monitor

- A process might access a resource without first gaining access permission to the resource.
- A process might never release a resource once it has been granted access to the resource.
- A process might attempt to release a resource that it never requested.
- A process might request the same resource twice

Resuming Processes within a Monitor

Solutions

Include the resource access operations within the ResourceAllocator monitor.

Check for two conditions to establish the correctness of this system

- user processes must always make their calls on the monitor in a **correct sequence**
- Ensure that an **uncooperative process does not simply ignore the mutual-exclusion gateway** provided by the monitor and try to access the shared resource directly, without using the access protocols